



Gateway: Audit Diff Doc

Audit Diff Document — EVM Gateway Contracts

Branches compared: `audit-main` (pre-audit) → `audit-main-fixes` (post-audit)

Purpose: Comprehensive listing of every contract code change made during the audit period. Organized by contract. Test-only changes are excluded.

Audit Diff Document — EVM Gateway Contracts

UniversalGateway.sol

1. Access Control Redesign — Granular role separation with AccessControlDefaultAdminRulesUpgradeable
2. _handleDeposits: Reject fee-on-transfer ERC20 tokens at deposit time
3. _swapToNative: Reject fee-on-transfer tokens in swap path
4. updateEpochDuration: Emit epoch index on duration change for auditability
5. setTSS → updateTSS: Atomic role transfer, no redundant TSS_ROLE in UG
6. setVault → updateVault: Atomic role transfer prevents desync
7. _checkBlockUSDCap: Accumulate raw wei to avoid mixed-price accounting
8. _routeUniversalTx: Infer TX_TYPE after fee deduction
9. sendUniversalTx (token overload): Reject non-zero msg.value in token-as-gas overload
10. sendUniversalTx (token overload): Remove redundant deadline pre-check
11. setChainlinkStalePeriod: Enforce minimum stale period
12. setProtocolFee → setInboundFee: Cap + rename
13. setRouters → updateUniswapV3Config: Rename + emit event
14. setCEAFactory → updateCEAFactory: Rename consistency
15. initialize: Reject half-configured Uniswap addresses
16. _validateRevertParams: Reject stray msg.value on ERC20 revert/rescue
17. _checkUSDCaps → checkUSDCaps: Align naming with visibility
18. swapToNative → _swapToNative: Align naming with visibility
19. _sendTxWithFunds Case 2.3: Remove redundant else-if condition

- 20. _routeUniversalTx terminal else: Document unreachable branch
- 21. _findV3PoolWithNative: Remove dead WETH branch
- 22. TSS_ROLE: Remove redundant TSS_ROLE management
- 23. State variable renames: camelCase for mutables
- 24. unpause: Role change
- 25. paused(): Expose paused state for Vault migration check
- 26. NatSpec: Correct misleading TSS control description
- 27. Interface changes (IUniversalGateway.sol)

UniversalGatewayPC.sol

- 1. Access Control Redesign — Granular role separation with AccessControlDefaultAdminRulesUpgradeable
- 2. _burnPRC20: Check transferFrom return value
- 3. sendUniversalTxOutbound: Validate token support via UniversalCore
- 4. sendUniversalTxOutbound: User-controlled swap slippage via maxPCForGas
- 5. _fetchOutboundTxGasAndFees: Delegate gas-limit resolution to UniversalCore
- 6. updateUniversalCore: Add admin setter for UniversalCore
- 7. setVaultPC → updateVaultPC: Rename consistency
- 8. State variable renames: camelCase + layout fix
- 9. unpause: Role change
- 10. Interface changes (IUniversalGatewayPC.sol)
- 11. TypesUGPC.sol — Added `maxPCForGas` field
- 12. IUniversalCore.sol — Interface changes

Vault.sol

- 1. Access Control Redesign — Granular role separation with AccessControlDefaultAdminRulesUpgradeable
- 2. TSS_ADDRESS removed: Remove redundant state variable
- 3. migrateTokens: Safe vault-to-vault token migration
- 4. receive(): Allow Vault to receive native ETH
- 5. setGateway → updateGateway: Rename consistency
- 6. setCEAFactory → updateCEAFactory: Rename consistency
- 7. unpause: Role change
- 8. Interface changes (IVault.sol)

VaultPC.sol

- 1. Access Control Redesign — Granular role separation with AccessControlDefaultAdminRulesUpgradeable
- 2. withdraw / withdrawToken: Include recipient in FeesWithdrawn event
- 3. withdraw / withdrawToken: Role change
- 4. unpause: Role change
- 5. Interface changes (IVaultPC.sol)

[Errors.sol \(Library\)](#)

[1. New errors added](#)

[Summary of Breaking Changes for Integrators](#)

[Function Renames \(call sites must update\)](#)

[State Variable Renames \(read accessors changed\)](#)

[Event Signature Changes](#)

[Struct Changes](#)

[New Functions](#)

[Role System Overhaul \(all contracts\)](#)

[IUniversalCore Interface Changes \(cross-repo dependency\)](#)

[Review Tasks](#)

UniversalGateway.sol

1. Access Control Redesign — Granular role separation with `AccessControlDefaultAdminRulesUpgradeable`

Full specification:  [Push Smart Contracts - AccessControl](#)

2. `_handleDeposits`: Reject fee-on-transfer ERC20 tokens at deposit time

Fee-on-transfer tokens could cause vault insolvency. Now a balance-before/after check rejects any token where the received amount differs from the transfer amount.

Before:

```
function _handleDeposits(address token, uint256 amount) internal {
    if (token == address(0)) {
        (bool ok,) = payable(TSS_ADDRESS).call{ value: amount }("");
        if (!ok) revert Errors.DepositFailed();
    } else {
        if (tokenToLimitThreshold[token] == 0) revert Errors.NotSupported();
    }
}
```

```

        IERC20(token).safeTransferFrom(_msgSender(), VAULT, a
mount);
    }
}

```

After:

```

function _handleDeposits(address token, uint256 amount) inter
nal {
    if (token == address(0)) {
        (bool ok,) = payable(tssAddress).call{ value: amount
}("");
        if (!ok) revert Errors.DepositFailed();
    } else {
        if (tokenToLimitThreshold[token] == 0) revert Errors.
NotSupported();
        uint256 balBefore = IERC20(token).balanceOf(vault);
        IERC20(token).safeTransferFrom(_msgSender(), vault, a
mount);
        uint256 received = IERC20(token).balanceOf(vault) - b
alBefore;
        if (received != amount) revert Errors.InvalidAmount
();
    }
}

```

ERC20 deposits now revert if the vault receives fewer tokens than expected (fee-on-transfer rejection).

3. **_swapToNative: Reject fee-on-transfer tokens in swap path**

Same fee-on-transfer protection applied to the token → native swap function.

Before:

```

IERC20(tokenIn).safeTransferFrom(_msgSender(), address(this),
amountIn);

```

```
IERC20(tokenIn).safeIncreaseAllowance(address(univ3Router), a
mountIn);
```

After:

```
uint256 balBefore = IERC20(tokenIn).balanceOf(address(this));
IERC20(tokenIn).safeTransferFrom(_msgSender(), address(this),
amountIn);
if (IERC20(tokenIn).balanceOf(address(this)) - balBefore != a
mountIn) revert Errors.InvalidAmount();
IERC20(tokenIn).safeIncreaseAllowance(address(univ3Router), a
mountIn);
```

Reverts if gateway receives fewer tokens than specified (prevents swap amount mismatches).

4. updateEpochDuration: Emit epoch index on duration change for auditability

Changing epoch duration implicitly resets all per-token rate-limit usage. The event now includes the epoch index at the time of change so off-chain monitors can detect the reset.

Before:

```
function updateEpochDuration(uint256 newDurationSec) external
onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
    if (newDurationSec == 0) revert Errors.InvalidInput();
    uint256 old = epochDurationSec;
    epochDurationSec = newDurationSec;
    emit EpochDurationUpdated(old, newDurationSec);
}
```

After:

```
function updateEpochDuration(uint256 newDurationSec) external
onlyRole(UG_ADMIN_ROLE) whenNotPaused {
```

```

    if (newDurationSec == 0) revert Errors.InvalidInput();
    uint256 old = epochDurationSec;
    uint64 epochIndexAtChange = uint64(block.timestamp / old);
    epochDurationSec = newDurationSec;
    emit EpochDurationUpdated(old, newDurationSec, epochIndexAtChange);
}

```

`EpochDurationUpdated` event signature changed: now includes third param `uint64 epochIndexAtChange`.

5. setTSS → updateTSS: Atomic role transfer, no redundant TSS_ROLE in UG

TSS role management removed from UG. Setter renamed for clarity.

Before:

```

function setTSS(address newTSS) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newTSS == address(0)) revert Errors.ZeroAddress();
    address old = TSS_ADDRESS;
    if (hasRole(TSS_ROLE, old)) _revokeRole(TSS_ROLE, old);
    _grantRole(TSS_ROLE, newTSS);
    TSS_ADDRESS = newTSS;
}

```

After:

```

function updateTSS(address newTSS) external onlyRole(OPERATOR_ROLE) {
    if (newTSS == address(0)) revert Errors.ZeroAddress();
    tssAddress = newTSS;
}

```

Renamed `setTSS` → `updateTSS`. Removed `TSS_ROLE` grant/revoke (UG never uses `TSS_ROLE` for authorization). Role changed from `DEFAULT_ADMIN_ROLE` to `OPERATOR_ROLE`.

6. `setVault` → `updateVault`: Atomic role transfer prevents desync

Before:

```
function setVault(address newVault) external onlyRole(DEFAULT_ADMIN_ROLE) {
    ...
    address old = VAULT;
    if (hasRole(VAULT_ROLE, old)) _revokeRole(VAULT_ROLE, old);
    _grantRole(VAULT_ROLE, newVault);
    VAULT = newVault;
    ...
}
```

After:

```
function updateVault(address newVault) external onlyRole(OPERATOR_ROLE) {
    ...
    address old = vault;
    if (hasRole(VAULT_ROLE, old)) _revokeRole(VAULT_ROLE, old);
    _grantRole(VAULT_ROLE, newVault);
    vault = newVault;
    ...
}
```

Renamed `setVault` → `updateVault`. Role changed from `DEFAULT_ADMIN_ROLE` to `OPERATOR_ROLE`.

7. `_checkBlockUSDCap`: Accumulate raw wei to avoid mixed-price accounting

Per-block USD cap previously summed USD values computed with potentially different oracle prices within a block. Now accumulates raw wei and converts the total at check time.

Before:

```
function _checkBlockUSDCap(uint256 amountWei) private {
    uint256 cap = BLOCK_USD_CAP;
    if (cap == 0) return;
    if (block.number != _lastBlockNumber) {
        _lastBlockNumber = block.number;
        _consumedUSDinBlock = 0;
    }
    uint256 usd1e18 = quoteEthAmountInUsd1e18(amountWei);
    if (usd1e18 > cap) revert Errors.BlockCapLimitExceeded();
    unchecked {
        uint256 newUsed = _consumedUSDinBlock + usd1e18;
        if (newUsed > cap) revert Errors.BlockCapLimitExceeded();
        _consumedUSDinBlock = newUsed;
    }
}
```

After:

```
function _checkBlockUSDCap(uint256 amountWei) private {
    uint256 cap = blockUsdCap;
    if (cap == 0) return;
    if (block.number != _lastBlockNumber) {
        _lastBlockNumber = block.number;
        _consumedWeiInBlock = 0;
    }
    uint256 newWei = _consumedWeiInBlock + amountWei;
    uint256 usdTotal = quoteEthAmountInUsd1e18(newWei);
    if (usdTotal > cap) revert Errors.BlockCapLimitExceeded();
}
```



```

    _consumedWeiInBlock = newWei;
}

```

Storage variable renamed `_consumedUSDinBlock` → `_consumedWeiInBlock`. Semantic changed from storing pre-converted USD to raw wei; USD conversion happens once at check time using the current oracle price.

8. `_routeUniversalTx`: Infer `TX_TYPE` after fee deduction

`TX_TYPE` was inferred from pre-fee `msg.value`, but routing used post-fee value. This caused zero-value GAS events when `msg.value == INBOUND_FEE`.

Before:

```

function sendUniversalTx(UniversalTxRequest calldata req) external payable nonReentrant whenNotPaused {
    if (!_isCallerCEA()) revert Errors.InvalidInput();
    uint256 nativeValue = msg.value;
    TX_TYPE txType = _fetchTxType(req, nativeValue);
    _routeUniversalTx(req, _msgSender(), nativeValue, txType, false);
}

function _routeUniversalTx(
    UniversalTxRequest memory req, address caller, uint256 nativeValue, TX_TYPE txType, bool fromCEA
) internal {
    ...
    (nativeValue, feeCollected) = _collectInboundFee(nativeValue);
    // txType already decided from pre-fee value
    ...
}

```

After:

```

function sendUniversalTx(UniversalTxRequest calldata req) external payable nonReentrant whenNotPaused {
    if (!_isCallerCEA()) revert Errors.InvalidInput();
    _routeUniversalTx(req, _msgSender(), msg.value, false);
}

function _routeUniversalTx(UniversalTxRequest memory req, address caller, uint256 nativeValue, bool fromCEA) internal {
    ...
    (nativeValue, feeCollected) = _collectInboundFee(nativeValue);
    TX_TYPE txType = _fetchTxType(req, nativeValue); // now post-fee
    ...
}

```

`TX_TYPE` is now inferred AFTER fee deduction, ensuring the inferred type matches the effective native value used downstream.

9. sendUniversalTx (token overload): Reject non-zero msg.value in token-as-gas overload

The token-gas overload is `payable` but derives `nativeValue` from the swap output. Any ETH sent as `msg.value` was permanently trapped.

Before:

```

function sendUniversalTx(UniversalTokenTxRequest calldata req Token) external payable nonReentrant whenNotPaused {
    if (!_isCallerCEA()) revert Errors.InvalidInput();
    if (reqToken.gasToken == address(0)) revert Errors.InvalidInput();
}

```

```
    ...  
}
```

After:

```
function sendUniversalTx(UniversalTokenTxRequest calldata req  
Token) external payable nonReentrant whenNotPaused {  
    if (!_isCallerCEA()) revert Errors.InvalidInput();  
    if (msg.value != 0) revert Errors.InvalidInput();  
    if (reqToken.gasToken == address(0)) revert Errors.InvalidInput();  
    ...  
}
```

Added `if (msg.value != 0) revert` at the start to prevent accidental ETH loss.

10. sendUniversalTx (token overload): Remove redundant deadline pre-check

Deadline validation existed in both the entrypoint and `_swapToNative`. Removed the duplicate.

Before:

```
if (reqToken.deadline != 0 && reqToken.deadline < block.timestamp)  
    revert Errors.SlippageExceededOrExpired();  
uint256 nativeValue = swapToNative(reqToken.gasToken, reqToken.gasAmount,  
    reqToken.amountOutMinETH, reqToken.deadline);
```

After:

```
uint256 nativeValue = _swapToNative(reqToken.gasToken, reqToken.gasAmount,  
    reqToken.amountOutMinETH, reqToken.deadline);
```

Redundant deadline check removed. `_swapToNative` is the single source of truth for deadline validation.

11. setChainlinkStalePeriod: Enforce minimum stale period

Setting `chainlinkStalePeriod = 0` disabled oracle freshness checks entirely. Now requires a minimum.

Before:

```
function setChainlinkStalePeriod(uint256 stalePeriodSec) external onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
    chainlinkStalePeriod = stalePeriodSec;
}
```

After:

```
uint256 public constant MIN_CHAINLINK_STALE_PERIOD = 10 minutes;

function setChainlinkStalePeriod(uint256 stalePeriodSec) external onlyRole(UG_ADMIN_ROLE) whenNotPaused {
    if (stalePeriodSec < MIN_CHAINLINK_STALE_PERIOD) revert Errors.InvalidInput();
    chainlinkStalePeriod = stalePeriodSec;
}
```

Added `MIN_CHAINLINK_STALE_PERIOD = 10 minutes` constant and floor check.

12. setProtocolFee → setInboundFee: Cap + rename

Fee setter had no upper bound and misleading name.

Before:

```
uint256 public INBOUND_FEE;

function setProtocolFee(uint256 fee) external onlyRole(DEFAULT_ADMIN_ROLE) {
    INBOUND_FEE = fee;
}
```

```
    emit ProtocolFeeUpdated(fee);
}
```

After:

```
uint256 public constant MAX_INBOUND_FEE = 0.05 ether;
uint256 public inboundFee;

function setInboundFee(uint256 fee) external onlyRole(UG_ADMIN_ROLE) {
    if (fee > MAX_INBOUND_FEE) revert Errors.InvalidInput();
    inboundFee = fee;
    emit ProtocolFeeUpdated(fee);
}
```

Renamed `setProtocolFee` → `setInboundFee`. Added `MAX_INBOUND_FEE = 0.05 ether` cap.
Variable renamed `INBOUND_FEE` → `inboundFee`.

13. setRouters → updateUniswapV3Config: Rename + emit event

Misleading plural name; no observability on changes.

Before:

```
function setRouters(address factory, address router) external
onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused {
    if (factory == address(0) || router == address(0)) revert
Errors.ZeroAddress();
    uniV3Factory = IUniswapV3Factory(factory);
    uniV3Router = ISwapRouterV3(router);
}
```

After:

```
function updateUniswapV3Config(address factory, address router) external
onlyRole(OPERATOR_ROLE) whenNotPaused {
    if (factory == address(0) || router == address(0)) revert
```

```

Errors.ZeroAddress();
    address oldFactory = address(uniV3Factory);
    address oldRouter = address(uniV3Router);
    uniV3Factory = IUniswapV3Factory(factory);
    uniV3Router = ISwapRouterV3(router);
    emit UniswapV3ConfigUpdated(oldFactory, factory, oldRoute
r, router);
}

```

Renamed `setRouters` → `updateUniswapV3Config`. Emits `UniswapV3ConfigUpdated` event. Role changed to `OPERATOR_ROLE`.

14. setCEAFactory → updateCEAFactory: Rename consistency

Before: `function setCEAFactory(address newFactory) external onlyRole(DEFAULT_ADMIN_ROLE)`

After: `function updateCEAFactory(address newFactory) external onlyRole(OPERATOR_ROLE)`

Renamed for consistency. Role changed to `OPERATOR_ROLE`.

15. initialize: Reject half-configured Uniswap addresses

Silently skipped Uniswap config if only one of factory/router was zero.

Before:

```

if (factory != address(0) && router != address(0)) {
    uniV3Factory = IUniswapV3Factory(factory);
    uniV3Router = ISwapRouterV3(router);
}

```

After:

```

if ((factory == address(0)) != (router == address(0))) {
    revert Errors.ZeroAddress();
}
if (factory != address(0)) {
    uniV3Factory = IUniswapV3Factory(factory);
}

```

```
uniV3Router = ISwapRouterV3(router);  
}
```

Reverts if exactly one of factory/router is zero. Both zero (disabled) or both non-zero (configured) are valid.

16. `_validateRevertParams`: Reject stray `msg.value` on ERC20 revert/rescue

ETH sent with an ERC20 revert call was permanently locked.

Before:

```
if (amount == 0 || (token == address(0) && msg.value != amount))  
    revert Errors.InvalidAmount();
```

After:

```
if (amount == 0) revert Errors.InvalidAmount();  
if (token == address(0)) {  
    if (msg.value != amount) revert Errors.InvalidAmount();  
} else {  
    if (msg.value != 0) revert Errors.InvalidAmount();  
}
```

Split validation: native path requires `msg.value == amount`; ERC20 path requires `msg.value == 0`.

17. `_checkUSDCaps` → `checkUSDCaps`: Align naming with visibility

Public function had private naming prefix.

Before: `function _checkUSDCaps(uint256 amount) public view`

After: `function checkUSDCaps(uint256 amount) public view`

Removed underscore prefix to match `public` visibility convention.

18. swapToNative → _swapToNative: Align naming with visibility

Internal function lacked private naming prefix.

Before: `function swapToNative(...) internal returns (uint256 ethOut)`

After: `function _swapToNative(...) internal returns (uint256 ethOut)`

Added underscore prefix to match `internal` visibility convention.

19. _sendTxWithFunds Case 2.3: Remove redundant else-if condition

Condition `_req.token != address(0)` was always true at that point.

Before:

```
else if (_req.token != address(0)) {
    uint256 gasAmount = nativeValue;
    ...
}
```

After:

```
else {
    uint256 gasAmount = nativeValue;
    ...
}
```

Changed `else if (_req.token != address(0))` to `else` (condition guaranteed by prior branches).

20. _routeUniversalTx terminal else: Document unreachable branch

Before:

```
else {
    revert Errors.InvalidTxType();
}
```



```
}
```

After:

```
// Defensive: unreachable under the current _fetchTxType logi  
c ...  
else {  
    revert Errors.InvalidTxType();  
}
```

Added defensive invariant comment. No code logic change.

21. _findV3PoolWithNative: Remove dead WETH branch

The `tokenIn == WETH` branch was unreachable because `_swapToNative` already handles WETH before calling this function.

Before:

```
function _findV3PoolWithNative(address tokenIn) internal view  
returns (IUniswapV3Pool pool, uint24 fee) {  
    if (tokenIn == address(0) || WETH == address(0)) revert E  
rrors.ZeroAddress();  
    if (tokenIn == WETH) {  
        return (IUniswapV3Pool(address(0)), 0);  
    }  
    ...  
}
```

After:

```
function _findV3PoolWithNative(address tokenIn) internal view  
returns (IUniswapV3Pool pool, uint24 fee) {  
    if (tokenIn == address(0) || weth == address(0)) revert E  
rrors.ZeroAddress();
```

```
...  
}
```

Removed unreachable `if (tokenIn == weth)` branch.

22. TSS_ROLE: Remove redundant TSS_ROLE management

`TSS_ROLE` was declared and managed in UG but never used for `onlyRole` authorization.

Before:

```
bytes32 public constant TSS_ROLE = keccak256("TSS_ROLE");  
// In initialize:  
_grantRole(TSS_ROLE, tss);  
// In setTSS:  
if (hasRole(TSS_ROLE, old)) _revokeRole(TSS_ROLE, old);  
_grantRole(TSS_ROLE, newTSS);
```

After:

`TSS_ROLE` constant removed entirely from UniversalGateway. No role grant/revoke in `initialize` or `updateTSS`. Only `tssAddress` state variable remains (used as fee/deposit recipient address, not for access control).

23. State variable renames: camelCase for mutables

All mutable ALL_CAPS state variables renamed to camelCase:

Old Name	New Name
<code>TSS_ADDRESS</code>	<code>tssAddress</code>
<code>VAULT</code>	<code>vault</code>
<code>BLOCK_USD_CAP</code>	<code>blockUsdCap</code>
<code>MIN_CAP_UNIVERSAL_TX_USD</code>	<code>minCapUniversalTxUsd</code>
<code>MAX_CAP_UNIVERSAL_TX_USD</code>	<code>maxCapUniversalTxUsd</code>
<code>WETH</code>	<code>weth</code>
<code>CEA_FACTORY</code>	<code>ceaFactory</code>

Old Name	New Name
<code>INBOUND_FEE</code>	<code>inboundFee</code>
<code>_consumedUSDinBlock</code>	<code>_consumedWeiInBlock</code>

Each variable now has an explicit NatSpec comment documenting it is admin-mutable.

24. unpause: Role change

Before: `function unpause() external whenPaused onlyRole(PAUSER_ROLE)`

After: `function unpause() external whenPaused onlyRole(OPERATOR_ROLE)`

`unpause` moved from `PAUSER_ROLE` to `OPERATOR_ROLE` (pause/unpause asymmetry by design).

25. paused(): Expose paused state for Vault migration check

Added:

```
function paused() public view override(PausableUpgradeable, I
UniversalGateway) returns (bool) {
    return super.paused();
}
```

Explicit override to expose `paused()` in the interface (used by `Vault.migrateTokens` to check gateway is paused).

26. NatSpec: Correct misleading TSS control description

Top-level contract NatSpec rewritten to accurately describe: (1) Vault-controlled revert/rescue (TSS enforcement upstream in Vault), (2) Token support list is for rate-limiting, not gas token validation.

27. Interface changes (IUniversalGateway.sol)

- `EpochDurationUpdated` event: added `uint64 epochIndexAtChange` parameter
- `UniversalTxExecuted` event: **removed entirely** (declared but never emitted)

- Added `UniswapV3ConfigUpdated` event
 - `INBOUND_FEE()` → `inboundFee()` in interface
 - `MIN_CAP_UNIVERSAL_TX_USD` / `MAX_CAP_UNIVERSAL_TX_USD` → `minCapUniversalTxUsd` / `maxCapUniversalTxUsd` in NatSpec
 - Added `paused() external view returns (bool)` to interface
 - NatSpec: `_checkUSDCaps` → `checkUSDCaps`
-

UniversalGatewayPC.sol

1. Access Control Redesign — Granular role separation with AccessControlDefaultAdminRulesUpgradeable

Full specification:  [Push Smart Contracts - AccessControl](#)

2. `_burnPRC20`: Check `transferFrom` return value

`transferFrom` return value was unchecked. A PRC20 returning `false` would silently proceed.

Before:

```
function _burnPRC20(address from, address token, uint256 amount) internal {
    IPRC20(token).transferFrom(from, address(this), amount);
    bool ok = IPRC20(token).burn(amount);
    if (!ok) revert Errors.TokenBurnFailed(token, amount);
}
```

After:

```
function _burnPRC20(address from, address token, uint256 amount) internal {
    bool transferred = IPRC20(token).transferFrom(from, address(this), amount);
    if (!transferred) revert Errors.TokenTransferFailed(token, amount);
}
```

```

n, amount);
    bool ok = IPRC20(token).burn(amount);
    if (!ok) revert Errors.TokenBurnFailed(token, amount);
}

```

Now checks and reverts on `false` return from `transferFrom`.

3. sendUniversalTxOutbound: Validate token support via UniversalCore

Unsupported/fake tokens could emit outbound events without validation.

Before:

```

function sendUniversalTxOutbound(UniversalOutboundTxRequest c
alldata req) ... {
    _validateParams(req.token, req.revertRecipient);
    TX_TYPE txType = _fetchTxType(req);
    ...
}

```

After:

```

function sendUniversalTxOutbound(UniversalOutboundTxRequest c
alldata req) ... {
    _validateParams(req.token, req.revertRecipient);
    if (!IUniversalCore(universalCore).isSupportedToken(req.t
oken)) revert Errors.NotSupported();
    TX_TYPE txType = _fetchTxType(req);
    ...
}

```

Added `isSupportedToken` check before processing.

4. sendUniversalTxOutbound: User-controlled swap slippage via maxPCForGas

Users had no control over how much native PC the gas-free swap could consume.

Before:

```
if (protocolFee > 0) {
    (bool ok,) = address(VAULT_PC).call{ value: protocolFee }
    ("");
    if (!ok) revert Errors.InvalidInput();
}
_swapAndCollectFees(gasToken, msg.value - protocolFee, gasFee);
```

After:

```
if (protocolFee > 0) {
    (bool ok,) = address(vaultPC).call{ value: protocolFee }
    ("");
    if (!ok) revert Errors.InvalidInput();
}
uint256 pcForSwap = msg.value - protocolFee;
if (req.maxPCForGas != 0) {
    if (req.maxPCForGas > pcForSwap) revert Errors.InvalidAmount();
    uint256 excess = pcForSwap - req.maxPCForGas;
    pcForSwap = req.maxPCForGas;
    if (excess > 0) {
        (bool refundOk,) = msg.sender.call{ value: excess }
        ("");
        if (!refundOk) revert Errors.WithdrawFailed();
    }
}
_swapAndCollectFees(gasToken, pcForSwap, gasFee);
```

When `maxPCForGas > 0`, UGPC caps native PC forwarded to UniversalCore and refunds excess before the swap. `maxPCForGas = 0` preserves old behavior (no cap).

5. `_fetchOutboundTxGasAndFees`: Delegate gas-limit resolution to `UniversalCore`

UGPC called `IUniversalCore(UNIVERSAL_CORE).BASE_GAS_LIMIT()` which doesn't exist on `UniversalCore`.

Before:

```
function _fetchOutboundTxGasAndFees(address token, uint256 gasLimit) internal view returns (...) {
    gasLimitUsed = gasLimit == 0 ? IUniversalCore(UNIVERSAL_CORE).BASE_GAS_LIMIT() : gasLimit;
    (gasToken, gasFee, protocolFee, gasPrice, chainNamespace)
    =
        IUniversalCore(UNIVERSAL_CORE).getOutboundTxGasAndFees(token, gasLimitUsed);
    ...
}
```

After:

```
function _fetchOutboundTxGasAndFees(address token, uint256 gasLimit) internal view returns (...) {
    (gasToken, gasFee, protocolFee, gasPrice, chainNamespace, gasLimitUsed) =
        IUniversalCore(universalCore).getOutboundTxGasAndFees(token, gasLimit);
    ...
}
```

Removed broken `BASE_GAS_LIMIT()` call. Passes `gasLimit` directly (even if 0). `UniversalCore` resolves defaults internally and returns `gasLimitUsed` as the 6th return value.

6. `updateUniversalCore`: Add admin setter for `UniversalCore`

No way to update `UNIVERSAL_CORE` without proxy upgrade.

Before: No setter existed.

After:

```
function updateUniversalCore(address _universalCore) external
onlyRole(OPERATOR_ROLE) whenNotPaused {
    if (_universalCore == address(0)) revert Errors.ZeroAddress();
    address oldUniversalCore = universalCore;
    universalCore = _universalCore;
    emit UniversalCoreUpdated(oldUniversalCore, _universalCore);
}
```

New function to update UniversalCore dependency without proxy upgrade.

7. setVaultPC → updateVaultPC: Rename consistency

Before: `function setVaultPC(address vaultPC) external onlyRole(DEFAULT_ADMIN_ROLE) whenNotPaused`

After: `function updateVaultPC(address _vaultPC) external onlyRole(OPERATOR_ROLE) whenNotPaused`

Renamed for consistency. Role changed to `OPERATOR_ROLE`.

8. State variable renames: camelCase + layout fix

Old Name	New Name
<code>UNIVERSAL_CORE</code>	<code>universalCore</code>
<code>VAULT_PC</code>	<code>vaultPC</code>

State variables grouped together above the constructor (previously split across it).

9. unpause: Role change

Before: `function unpause() external onlyRole(PAUSER_ROLE) whenPaused`

After: `function unpause() external onlyRole(OPERATOR_ROLE) whenPaused`

10. Interface changes (IUniversalGatewayPC.sol)

- Added `UniversalCoreUpdated` event
 - `UNIVERSAL_CORE()` → `universalCore()` in interface
 - NatSpec: added documentation for `maxPCForGas` slippage control
-

11. TypesUGPC.sol — Added `maxPCForGas` field

Before:

```
struct UniversalOutboundTxRequest {
    bytes    recipient;
    address token;
    uint256 amount;
    uint256 gasLimit;                // gas limit for fee quo
te; 0 = default BASE_GAS_LIMIT
    bytes    payload;
    address revertRecipient;
}
```

After:

```
struct UniversalOutboundTxRequest {
    bytes    recipient;
    address token;
    uint256 amount;
    uint256 gasLimit;                // gas limit for fee quo
te; 0 = per-chain default
    uint256 maxPCForGas;            // max native PC for gas
swap; 0 = no cap
    bytes    payload;
    address revertRecipient;
}
```

New field `maxPCForGas` inserted between `gasLimit` and `payload` .

12. IUniversalCore.sol — Interface changes

- **Removed:** `function BASE_GAS_LIMIT() external view returns (uint256 baseGasLimit)`
- **Modified:** `getOutboundTxGasAndFees` return tuple expanded from 5 to 6 values (added `uint256 gasLimitUsed`)
- **Added:** `function isSupportedToken(address token) external view returns (bool supported)`

Vault.sol

1. Access Control Redesign — Granular role separation with AccessControlDefaultAdminRulesUpgradeable

Full specification:  [Push Smart Contracts - AccessControl](#)

2. TSS_ADDRESS removed: Remove redundant state variable

`TSS_ADDRESS` was a convenience variable not used for authorization (only `hasRole(TSS_ROLE, msg.sender)` was used).

Before:

```
address public TSS_ADDRESS;
// In initialize:
TSS_ADDRESS = tss;
// setTSS existed:
function setTSS(address newTss) external onlyRole(DEFAULT_ADMIN_ROLE) {
    address old = TSS_ADDRESS;
    if (hasRole(TSS_ROLE, old)) _revokeRole(TSS_ROLE, old);
    _grantRole(TSS_ROLE, newTss);
    TSS_ADDRESS = newTss;
    emit TSSUpdated(old, newTss);
}
```

After:

`TSS_ADDRESS` state variable removed. `setTSS` function removed. `TSSUpdated` event removed from IVault. TSS is now managed purely via role grants/revokes through `ROLE_MANAGER_ROLE`.

3. migrateTokens: Safe vault-to-vault token migration

`setVault` (now `updateVault`) could orphan ERC20 funds in the old vault.

Added:

```
function migrateTokens(
    address newVault,
    address[] calldata tokens
) external nonReentrant whenPaused onlyRole(VAULT_ADMIN_ROLE)
{
    if (newVault == address(0)) revert Errors.ZeroAddress();
    if (tokens.length == 0) revert Errors.EmptyTokenList();
    if (!gateway.paused()) revert Errors.GatewayNotPaused();

    uint256 len = tokens.length;
    uint256[] memory amounts = new uint256[](len);

    for (uint256 i; i < len; ++i) {
        uint256 bal = IERC20(tokens[i]).balanceOf(address(this));
        amounts[i] = bal;
        if (bal > 0) {
            IERC20(tokens[i]).safeTransfer(newVault, bal);
        }
    }

    uint256 nativeBal = address(this).balance;
    if (nativeBal > 0) {
        (bool ok,) = newVault.call{ value: nativeBal }("");
        if (!ok) revert Errors.WithdrawFailed();
    }
}
```

```
    emit TokensMigrated(newVault, tokens, amounts, nativeBa  
  1);  
}
```

New function for safe ERC20 + native migration. Requires BOTH vault and gateway to be paused. Gated by `VAULT_ADMIN_ROLE`.

4. receive(): Allow Vault to receive native ETH

Added:

```
receive() external payable {}
```

Required for vault migration (receiving native ETH from old vault).

5. setGateway → updateGateway: Rename consistency

Before: `function setGateway(address gw) external onlyRole(DEFAULT_ADMIN_ROLE)`

After: `function updateGateway(address gw) external onlyRole(OPERATOR_ROLE)`

6. setCEAFactory → updateCEAFactory: Rename consistency

Before: `function setCEAFactory(address newCEAFactory) external onlyRole(DEFAULT_ADMIN_ROLE)`

After: `function updateCEAFactory(address newCEAFactory) external onlyRole(OPERATOR_ROLE)`

7. unpause: Role change

Before: `function unpause() external whenPaused onlyRole(PAUSER_ROLE)`

After: `function unpause() external whenPaused onlyRole(OPERATOR_ROLE)`

8. Interface changes (IVault.sol)

- **Removed:** `TSSUpdated` event
- **Added:** `TokensMigrated` event
- **Added:** `migrateTokens(address newVault, address[] calldata tokens) external` function

VaultPC.sol

1. Access Control Redesign — Granular role separation with AccessControlDefaultAdminRulesUpgradeable

Full specification:  [Push Smart Contracts - AccessControl](#)

2. withdraw / withdrawToken: Include recipient in FeesWithdrawn event

Event omitted the `to` recipient address.

Before:

```
emit FeesWithdrawn(msg.sender, address(0), amount);    // with
draw
emit FeesWithdrawn(msg.sender, token, amount);          // wit
hdrawToken
```

After:

```
emit FeesWithdrawn(msg.sender, to, address(0), amount); //
withdraw
emit FeesWithdrawn(msg.sender, to, token, amount);      //
withdrawToken
```

`FeesWithdrawn` event signature changed: `(address indexed caller, address indexed to, address indexed token, uint256 amount)`.

3. withdraw / withdrawToken: Role change

Before: `onlyRole(MANAGER_ROLE)`

After: `onlyRole(VPC_ADMIN_ROLE)`

4. unpause: Role change

Before: `function unpause() external whenPaused onlyRole(PAUSER_ROLE)`

After: `function unpause() external whenPaused onlyRole(OPERATOR_ROLE)`

5. Interface changes (IVaultPC.sol)

- `FeesWithdrawn` event: added `address indexed to` parameter (now 4 params instead of 3)
 - `MANAGER_ROLE` references → `VPC_ADMIN_ROLE` in NatSpec
-

Errors.sol (Library)

1. New errors added

```
error GatewayNotPaused();           // Used by Vault.migrateTokens
error EmptyTokenList();             // Used by Vault.migrateTokens
error TokenTransferFailed(address token, uint256 amount); // Used by UGPC._burnPRC20
```

Summary of Breaking Changes for Integrators

Function Renames (call sites must update)

Contract	Old Name	New Name
UniversalGateway	<code>setTSS</code>	<code>updateTSS</code>
UniversalGateway	<code>setVault</code>	<code>updateVault</code>
UniversalGateway	<code>setRouters</code>	<code>updateUniswapV3Config</code>
UniversalGateway	<code>setCEAFactory</code>	<code>updateCEAFactory</code>
UniversalGateway	<code>setProtocolFee</code>	<code>setInboundFee</code>
UniversalGatewayPC	<code>setVaultPC</code>	<code>updateVaultPC</code>
Vault	<code>setGateway</code>	<code>updateGateway</code>
Vault	<code>setCEAFactory</code>	<code>updateCEAFactory</code>

Contract	Old Name	New Name
Vault	<code>setTSS</code>	Removed (use role management)

State Variable Renames (read accessors changed)

Contract	Old Accessor	New Accessor
UniversalGateway	<code>TSS_ADDRESS()</code>	<code>tssAddress()</code>
UniversalGateway	<code>VAULT()</code>	<code>vault()</code>
UniversalGateway	<code>BLOCK_USD_CAP()</code>	<code>blockUsdCap()</code>
UniversalGateway	<code>MIN_CAP_UNIVERSAL_TX_USD()</code>	<code>minCapUniversalTxUsd()</code>
UniversalGateway	<code>MAX_CAP_UNIVERSAL_TX_USD()</code>	<code>maxCapUniversalTxUsd()</code>
UniversalGateway	<code>WETH()</code>	<code>weth()</code>
UniversalGateway	<code>CEA_FACTORY()</code>	<code>ceaFactory()</code>
UniversalGateway	<code>INBOUND_FEE()</code>	<code>inboundFee()</code>
UniversalGatewayPC	<code>UNIVERSAL_CORE()</code>	<code>universalCore()</code>
UniversalGatewayPC	<code>VAULT_PC()</code>	<code>vaultPC()</code>
Vault	<code>TSS_ADDRESS()</code>	Removed

Event Signature Changes

Contract	Event	Change
UniversalGateway	<code>EpochDurationUpdated</code>	Added <code>uint64 epochIndexAtChange</code> (3rd param)
UniversalGateway	<code>UniversalTxExecuted</code>	Removed entirely
UniversalGateway	<code>UniswapV3ConfigUpdated</code>	New event
VaultPC	<code>FeesWithdrawn</code>	Added <code>address indexed to</code> (2nd param, shifted others)
Vault	<code>TSSUpdated</code>	Removed
Vault	<code>TokensMigrated</code>	New event
UniversalGatewayPC	<code>UniversalCoreUpdated</code>	New event

Struct Changes

Struct	Change
<code>UniversalOutboundTxRequest</code>	New field <code>uint256 maxPCForGas</code> between <code>gasLimit</code> and <code>payload</code>

New Functions

Contract	Function	Purpose
UniversalGatewayPC	<code>updateUniversalCore(address)</code>	Admin setter for UniversalCore
Vault	<code>migrateTokens(address, address[])</code>	Safe vault-to-vault migration
UniversalGateway	<code>paused() returns (bool)</code>	Exposed in interface

Role System Overhaul (all contracts)

Full specification:  [Push Smart Contracts - AccessControl](#)

IUniversalCore Interface Changes (cross-repo dependency)

Change	Detail
Removed	<code>BASE_GAS_LIMIT() → uint256</code>
Modified	<code>getOutboundTxGasAndFees</code> returns 6 values (added <code>uint256 gasLimitUsed</code>)
Added	<code>isSupportedToken(address) → bool</code>

Review Tasks

Contract Name	Function	What to check	Owner
UniversalGateway	<code>_routeUniversalTx</code>	Test all TX_TYPE paths w/ inbound fee behave correctly post change	Shoaib Mohammed

Contract Name	Function	What to check	Owner
UniversalGateway	sendUniversalTx (token overload)	Tests: reject any non-zero msg.value in token-as-gas overload	Shoaib Mohammed
UniversalGatewayPC	sendUniversalTxOutbound	SDK impact: outbound payload struct param addition (maxPCForGas) doesn't break integrations	Shoaib Mohammed, Nilesh Gupta
UniversalGatewayPC	_fetchOutboundTxGasAndFees	Outbound tests: gas+fee quoting/return values match expected behavior	Aman Gupta, Shoaib Mohammed, Nilesh Gupta